SAMSUNG SW AI ACADEMY FOR YOUTH

對外秘

프로젝트 명세서

멀티모달 인식 통합과 VLM 기반

의도 파악 AI Workflow 개발

(AI 코어 프로젝트)

“PoC 제작 및 Object Detection 파인튜닝”

1 회차

對外秘

목차

1. PoC 제작 및 Object Detection 파인튜닝 3

1.1 목표 3

1.2 준비사항 3

1.3 작업 순서 4

1.4 요구사항 4

1.5 참고자료 7

1.6 프로젝트 결과 8

1. PoC 제작 및 Object Detection 파인튜닝

1.1 목표

이번 'PoC 제작 및 Object Detection 파인튜닝' 프로젝트의 목표는 다음과 같습니다.

- 정확도·속도·메모리·라이선스 기준으로 Object Detection 모델과 VLM 모델을 선정한다.
- 선정한 YOLO11n 과 Qwen2.5-VL-3B 를 직접 실행하고 추론 시간·메모리를 기록한다.
- YOLO 와 VLM 을 하나의 FastAPI 서버로 통합하는 데모 PoC 를 제작한다.
- 실제 운영 환경을 반영한 도메인 특화 데이터로 Object Detection 모델을 파인튜닝하고, 파인튜닝 전후의 성능을 정량적으로 비교·분석한다.

1.2 준비사항

1) 프로젝트 구조

- 이번 프로젝트는 카메라 영상과 사용자 질문을 함께 이해하는 멀티모달 AI Workflow 중 '인식' 단계를 먼저 만드는 구조로 진행한다. 개수는 Object Detection(YOLO), 장면의 맥락은 VLM 이 맡도록 역할을 나누고, 두 모델의 출력을 모두 텍스트로 통일한다.

  먼저 실습 환경을 설치하고 YOLO11n 과 Qwen2.5-VL-3B 를 각각 테스트해 성능을 기록한다. 이후 두 모델을 FastAPI 서버 하나로 묶어 PoC 를 완성하고, 데모 데이터셋으로 YOLO 를 파인튜닝해 전/후 결과를 비교한다.

  - ■ Python 3.12 가상환경·패키지·모델 가중치 설치 및 환경 체크
  - ■ Object Detection 후보 모델 비교 및 YOLO11n 선정
  - ■ YOLO 첫 추론, conf 임계값 비교, 성능 측정 기록표 작성
  - ■ VLM(Qwen2.5-VL-3B) 로드, 이미지 설명, 프롬프트별 출력 비교
  - ■ YOLO + VLM PoC FastAPI 서버 구현 및 순차·병렬 호출 측정
  - ■ 데이터셋 준비·검증, 파인튜닝, 전/후 평가 비교

2) 사용 데이터

- 이번 프로젝트에서는 별도의 대규모 학습 데이터셋을 직접 구축하지 않는다. 제공되는 샘플 이미지(sample.jpg)로 모델을 테스트하고, 파인튜닝은 실습용 데모 데이터셋(train 40 장·val 10 장, obstacle·marker 2 개 클래스)으로 라벨링 → 학습 → 평가의 흐름을 체험한다.

3) 개발언어 및 툴

- Python 3.12 : 모델 추론, 성능 측정, FastAPI 서버, 파인튜닝 코드를 실행하는 데 사용
- VSCode : 사전 배포된 실습 코드를 열어 읽고 수정·실행

4) 필수 라이브러리 / 오픈소스

- Ultralytics (YOLO11n) : Object Detection 추론 및 파인튜닝
- llama-cpp-python (0.3.34) : GGUF 양자화 VLM(Qwen2.5-VL-3B-Instruct) CPU 추론
- FastAPI / Uvicorn : YOLO·VLM 결과를 제공하는 PoC 서버
- huggingface_hub : hf download 명령으로 모델 가중치 다운로드

1.3 작업 순서

1) 실습 환경(가상환경·패키지·모델 가중치)을 설치하고 환경 체크 스크립트로 확인한다

2) Object Detection 후보 모델을 비교하고 YOLO11n 으로 추론·성능을 측정한다

3) VLM 을 로드하고 프롬프트에 따른 이미지 설명 차이를 비교한다

4) YOLO 와 VLM 을 하나의 FastAPI 서버로 통합하고 호출 시간을 측정한다

5) 데모 데이터셋을 준비·검증하고 YOLO 를 파인튜닝한다

6) 파인튜닝 전/후 모델을 같은 val 셋으로 평가해 비교한다

1.4 요구사항

본 프로젝트는 멀티모달 인식 모델의 선정 기준과 성능 측정, PoC 통합, 파인튜닝의 전 과정을 단계적으로 학습하는 데 목적이 있다.

■ **요구사항 예시 (참고용)**

- 아래의 내용을 참고하여 추가적인 아이디어에 대해 요구사항을 추가 또는 수정하여 기능을 구현합니다. 단, 필수 기능은 반드시 구현해야 하며 수정할 수 없습니다.

기능적 요구사항				
번호	분류	요구사항명	요구사항 상세	우선순위
F101	기능	Object Detection 모델 선정 및 성능 측정	YOLO11n 추론과 conf 임계값 비교, 성능 기록표 작성	필수
F102	기능	VLM 모델 선정 및 이미지 설명	Qwen2.5-VL-3B 로드와 프롬프트별 출력 비교	필수
F103	기능	YOLO + VLM 데모 PoC 서버 제작	이미지 1장으로 객체 요약과 장면 설명을 받는 FastAPI 서버	필수
F104	기능	Object Detection 모델 파인튜닝	데이터셋 준비, 전이학습, 파인튜닝 전/후 평가 비교	필수
F105	기능	운영 환경 맞춤 데이터셋 구축과 파인튜닝 개선	직접 촬영·라벨링한 데이터로 실제 임계값(conf 0.25)에서 동작하는 모델 만들기	심화
F106	기능	PoC 서버 응답 시간 최적화	병목(VLM)을 측정으로 찾고 입력 축소·출력 제한으로 응답 시간 단축	심화

1. 기본 기능 (필수)

1) Object Detection 모델 선정 및 성능 측정

① 요구사항 번호 : F101

② 정확도·속도·메모리·라이선스 기준으로 후보 모델을 비교하고, YOLO11n 의 선정 근거를 정리한다.

③ 샘플 이미지를 추론해 bbox·class·confidence 를 확인하고, conf 0.10 / 0.25 / 0.60 의 검출 결과를 비교한다.

④ 추론 시간·FPS·메모리 증가량을 측정해 measurement.csv 로 기록한다.

2) VLM 모델 선정 및 이미지 설명

① 요구사항 번호 : F102

② VLM 본체(GGUF Q4_K_M)와 mmproj 두 파일을 로드해 이미지를 한국어로 설명한다.

③ 같은 이미지에 프롬프트를 바꿔 언어·길이·내용의 차이를 비교하고, temperature=0.0 으로 재현성을 확보한다.

3) YOLO + VLM 데모 PoC 서버 제작

① 요구사항 번호 : F103

② /api/yolo-detect·/api/vlm-describe 두 엔드포인트를 '이미지 in → 텍스트 out' 형식으로 통일한다.

③ 모델은 지연 로딩 + 락으로 한 번만 올리고, 추론은 run_in_threadpool 로 위임해 서버가 멈추지 않게 한다.

④ 순차·병렬 호출의 소요 시간을 측정해 병목 구간을 확인한다.

4) Object Detection 모델 파인튜닝

① 요구사항 번호 : F104

② YOLO 형식 데이터셋(이미지 + 0~1 정규화 라벨)을 준비하고 라벨 범위·짝 누락·클래스 분포를 검증한다.

③ 사전학습 가중치(yolo11n.pt)에서 출발해 파인튜닝하고, 같은 val 셋으로 mAP50·precision·recall·max_conf 를 전/후 비교한다.

2. 심화 기능 (선택)

필수 기능을 모두 구현한 후에는 프로젝트를 한 단계 확장하는 아래 심화 기능에 도전해보자.

1) 운영 환경 맞춤 데이터셋 구축과 파인튜닝 개선

① 요구사항 번호 : F105

② 실제 운용과 같은 조건(웹캠·각도·조명)으로 장애물·표식 이미지를 직접 촬영하고, 라벨링 도구(Roboflow·labelImg·CVAT)로 YOLO 형식 라벨을 만들어 데이터셋을 구축한다 (클래스당 100 장 이상, 물체가 없는 배경 이미지 포함).

③ 데이터셋 검증 스크립트로 라벨 범위·짝 누락·클래스 분포를 확인한 뒤,

epochs·imgsz 를 한 번에 하나씩 바꿔 가며 파인튜닝한다.

④ 운영 임계값 conf 0.25 로 평가해 max_conf 가 0.25 를 넘고 precision·recall 이 함께 확보되는지 확인하고, 실험별 결과를 표로 정리한다.

2) PoC 서버 응답 시간 최적화

① 요구사항 번호 : F106

② 순차·병렬 호출 측정으로 VLM 이 전체 시간의 대부분을 차지하는 단일 병목임을 확인한다.

③ VLM 입력 이미지의 긴 변을 줄이고(예: 448px) max_tokens 를 제한하는 설정을 PoC 서버에 추가한다.

④ 설정별(원본 / 출력 제한 / 입력 축소) VLM 응답 시간과 설명 품질을 같은 이미지로 측정해 CSV 로 기록하고, 서버에 적용할 설정을 근거와 함께 선택한다.

1.5 참고자료

- Ultralytics YOLO11 공식 문서
  https://docs.ultralytics.com/models/yolo11/
- Ultralytics 학습(Train) 모드 문서
  https://docs.ultralytics.com/modes/train/
- Qwen2.5-VL-3B-Instruct GGUF 모델
  https://huggingface.co/ggml-org/Qwen2.5-VL-3B-Instruct-GGUF
- llama-cpp-python 공식 저장소
  https://github.com/abetlen/llama-cpp-python
- FastAPI 공식 문서
  https://fastapi.tiangolo.com/

API 명세				
번호	메서드	경로	요청 → 응답(200) / 오류	비고
A1	GET	/	대시보드 화면(index.html)을 돌려준다	2회차에서 제작 예정
A2	GET	/mediapipe/...	브라우저 제스처 인식 파일(vision_bundle.mjs, wasm, gesture_recognizer.task) / 404	2회차에서 제작 예정
A3	POST	/api/yolo-detect	JPEG 바이트 → {"summary": "person 4개, bus 1개", "elapsed_ms"} / 400 {"error": 메시지}	1회차에서 제작 완료
A4	POST	/api/vlm-describe	JPEG 바이트 → {"description", "elapsed_ms"} / 400 {"error": 메시지}	1회차에서 제작 완료
A5	POST	/api/llm-answer	인식 결과 Text 와 질문 → LLM 답변	2회차에서 제작 예정

1.6 프로젝트 결과

최종적으로 제출해야 할 항목은 아래와 같고 이어지는 프로젝트와 함께 PPTX 형식의 자료와 동작 영상을 촬영 하여 제출한다

■ **산출물과 제출**

1) PoC 서버의 YOLO·VLM 호출 결과와 성능 기록표(measurement.csv), 파인튜닝 전/후 비교표를 제출한다.

2) 최종 제출 시 파일의 명칭은 아래의 형식을 엄격히 준수하여 업로드한다.

◆ AI 코어_2 회차_PJT01_동작시연영상_지역_반_성명 1_성명 2.zip

- 以上